

架构候选方案对比与辩论记录

1. 项目背景与约束

本项目为“校园互助服务平台”，目标用户是在校大学生。根据 P1 需求规格说明书，系统需要支持柔性实名认证、多类型互助任务、匿名树洞、找搭子、积分激励、评价仲裁、消息通知和管理员审核等功能。

本阶段做架构选择时采用以下约束：

- 开发周期：10 周内完成可演示版本。
- 团队规模：4 人左右，后续维护成本需要可控。
- 用户规模：校园范围，饭点高峰期预计 500 人同时在线。
- 质量属性重点：安全与隐私、可维护性、开发效率、移动端适配、可观测性。
- 现有技术基础：需求文档已倾向 Vue 3 + Spring Boot + MySQL + Redis + JWT，后端构建产物中已有 Controller、Service、Mapper、Entity 分层结构。

2. 候选架构方案对比

对比维度	方案 A：前后端分离 + 分层单体	方案 B：微服务架构	方案 C：事件驱动架构
开发复杂度	中等。前端、后端、数据库边界清晰，后端内部按 Controller / Service / Mapper 分层即可。	高。需要服务拆分、服务发现、接口治理、跨服务事务、链路追踪。	中高。需要事件模型、消息队列、幂等处理、失败重试机制。
可维护性	高。模块可按用户、任务、社区、评价、消息、管理划分，代码集中，调试简单。	中高。理论上服务自治，但小团队容易因拆分过早造成接口维护成本上升。	中。异步解耦明显，但事件流不直观，排查问题成本较高。
可扩展性	中。单体可通过模块化、缓存、数据库索引和水平扩容支撑校园级规模。	高。可针对高负载服务独立扩容。	高。消息通知、积分结算、审核任务可异步扩展。
性能	中高。校园级并发可通过 Redis 缓存、分页、索引优化和静态资源缓存满足。	高。但服务间调用会增加网络开销。	高。异步任务能削峰，但核心交易仍需同步一致。

对比维度	方案 A：前后端分离 + 分层单体	方案 B：微服务架构	方案 C：事件驱动架构
团队学习成本	低。Vue、Spring Boot、MySQL、Redis 都是常见课程项目栈。	高。需要掌握 Spring Cloud 或类似体系。	中高。需要掌握消息队列、事件一致性、死信队列等。
10 周内可行性	高。能优先交付核心功能，并保留模块演进空间。	低。基础设施工作会挤压业务开发和测试时间。	中。可在局部使用事件思想，但不适合作为主架构。
与需求匹配度	高。任务交易、社区、评价仲裁都适合先以同步 REST API 实现。	中。系统边界尚未稳定，过早拆分会放大变更成本。	中。消息通知、积分记录适合异步，但发布接单、评价仲裁需要强一致流程。

3. 架构辩论赛记录

成员 A：直接询问推荐架构

Prompt

我们要做一个校园互助服务平台，支持学生发布和接单任务、二手交易、找搭子、匿名树洞、评价仲裁和消息通知。推荐使用什么架构？

AI 回答摘要

AI 推荐“前后端分离 + MVC / 分层架构”，理由是团队容易实现，适合 Web 和移动端访问，后端可以按业务模块拆分。AI 同时提到如果未来用户量扩大，可以演进到微服务。

人工评价

回答方向基本正确，但缺少对项目周期、团队规模、隐私约束和校园高峰场景的分析。它给出的是通用课程项目答案，不能直接作为最终决策依据。

成员 B：提供详细项目约束后询问

Prompt

我们团队 4 人，课程项目周期 10 周，系统是校园互助服务平台。需求包括柔性实名认证、匿名树洞、多类型任务发布、积分激励、评价仲裁、管理员审核。预计校园内 500 人高峰在线。技术基础是 Vue 3、

Spring Boot、MySQL、Redis。请在开发复杂度、可维护性、性能、安全、10 周可行性之间权衡，推荐架构。

AI 回答摘要

AI 推荐“前后端分离 + 模块化分层单体”，明确不建议第一版使用微服务。它建议后端按用户认证、任务、社区互动、评价仲裁、消息通知、后台管理划分模块，并使用 Redis 处理会话、热点任务列表或通知未读数缓存。

人工评价

该回答质量最高。它引用了团队人数、时间和技术栈，能把“理论上更先进”与“课程项目可落地”区分开。

成员 C：要求对比多种方案

Prompt

请对校园互助服务平台的 MVC / 分层架构、微服务架构、事件驱动架构做对比，说明每种方案的优缺点、适用条件，以及我们在 10 周课程项目中应该如何选择。

AI 回答摘要

AI 对比了三类架构。MVC / 分层架构实现简单、维护直观；微服务适合大团队和复杂业务，但部署运维成本高；事件驱动适合通知、积分、审核等异步流程，但会引入消息一致性和调试成本。AI 建议主架构采用分层单体，对消息通知和积分记录保留事件化接口。

人工评价

回答有助于辩论，因为它把“为什么不选其他方案”说清楚了。但对本项目已有数据库表和接口边界关注不足，需要人工补充模块依赖和数据一致性约束。

成员 D：让 AI 扮演架构师

Prompt

请扮演一名有 10 年经验的软件架构师，审查我们的校园互助平台需求。你需要特别关注隐私、安全、10 周落地和后续可扩展性，给出架构建议、模块划分和关键风险。

AI 回答摘要

AI 建议采用“前后端分离 + 后端模块化 + REST API + JWT 鉴权”。它强调匿名树洞不能和真实身份展示逻辑混在一起，建议在身份模块中保存真实认证信息，在社区模块中使用匿名展示身份。它也提示支付、学校 CAS、AI 内容审核等外部系统应设计适配层，避免业务代码直接绑定第三方实现。

人工评价

该回答对隐私和外部系统隔离的提醒很有价值。其不足是提出了过多非首版功能，例如完整支付托管和复杂审核 workflow，需要结合课程周期删减。

4. 团队辩论结论

最终选择：前后端分离 + 模块化分层单体架构。

选择理由：

- 与 10 周课程周期匹配，能优先交付注册登录、任务发布接单、社区互动、评价仲裁和后台管理。
- 与现有技术栈一致，前端使用 Vue 3，后端使用 Spring Boot，数据库使用 MySQL，缓存使用 Redis。
- 分层结构足以支撑校园级并发，性能瓶颈可通过索引、分页、缓存、静态资源缓存和接口限流处理。
- 后端可按业务模块组织代码，未来如通知、支付、审核成为高负载模块，可以再拆分为独立服务。
- 对匿名树洞、柔性实名、评价仲裁等需求，分层单体更容易保证事务一致性和权限校验一致性。

不选择微服务的原因：

- 当前业务边界还在变化，过早拆分会导致接口返工。
- 团队规模小，服务治理、部署、监控、链路追踪成本过高。
- 系统目标是校园范围，不需要一开始为互联网级流量付出复杂度成本。

不选择事件驱动作为主架构的原因：

- 任务发布、接单、完成、评价、仲裁存在明确的同步状态流转，必须保证用户立即获得确定结果。
- 消息队列会引入幂等、补偿、死信处理和事件追踪成本。
- 可以局部借鉴事件思想，例如“任务完成后生成积分记录”“评论后更新未读消息”，但第一版不把消息队列作为核心依赖。

5. 投票记录

成员	推荐方案	主要理由
成员 A	分层单体	实现成本低，课程项目风险小。
成员 B	分层单体	最符合团队人数、周期和技术栈。
成员 C	分层单体，局部保留事件化扩展点	兼顾可落地和后续演进。
成员 D	分层单体	隐私、权限和事务一致性更容易控制。

结论：4 票同意采用“前后端分离 + 模块化分层单体架构”。